

ImmoSN — Plateforme Immobilière Sénégalaise

Plateforme SaaS de gestion immobilière dédiée au marché sénégalais.
Elle permet la publication d'annonces, la gestion des prospects (leads), la messagerie entre clients et agents, la planification de visites, la signature de contrats et le suivi des litiges.

Table des matières

- 1. [Vue d'ensemble](#)
- 2. [Stack technique](#)
- 3. [Architecture globale](#)
- 4. [Prérequis](#)
- 5. [Démarrage rapide \(développement\)](#)
- 6. [Démarrage en production](#)
- 7. [Variables d'environnement](#)
- 8. [Structure du projet](#)
- 9. [Modèles de données](#)
- 10. [API REST — Endpoints](#)
- 11. [Sécurité et authentification](#)
- 12. [Flux métier](#)
- 13. [Frontend — Vue.js](#)
- 14. [Géolocalisation](#)
- 15. [Déploiement Docker](#)
- 16. [Configuration Nginx](#)

1. Vue d'ensemble

ImmoSN est une application web full-stack organisée en deux services :

Service	Technologie	Rôle
Backend	Spring Boot 4.0.6 / Java 21	API REST, logique métier, sécurité
Frontend	Vue 3 / Vite	Interface utilisateur SPA
Base de données	PostgreSQL (Neon) / H2 (dev)	Persistance des données
Proxy	Nginx	Reverse proxy + serveur statique

Rôles utilisateurs

Rôle	Accès
CLIENT	Navigation annonces, messagerie, visites, contrats, favoris, signalements
ADMIN	Gestion annonces, leads, visites, contrats, messages, dashboard

Rôle	Accès
<code>SUPER_ADMIN</code>	Tout ADMIN + gestion des comptes administrateurs

2. Stack technique

Backend

- **Framework** : Spring Boot 4.0.6
- **Langage** : Java 21
- **ORM** : Spring Data JPA / Hibernate
- **Sécurité** : Spring Security 6 + JWT (JJWT 0.11.5)
- **Validation** : Jakarta Bean Validation
- **Build** : Maven 3.9
- **Base de données** : PostgreSQL (prod) / H2 (dev)
- **Scheduler** : Spring `@Scheduled` (expiration automatique des contrats)
- **HTTP Client** : RestTemplate (géocodage Nominatim)

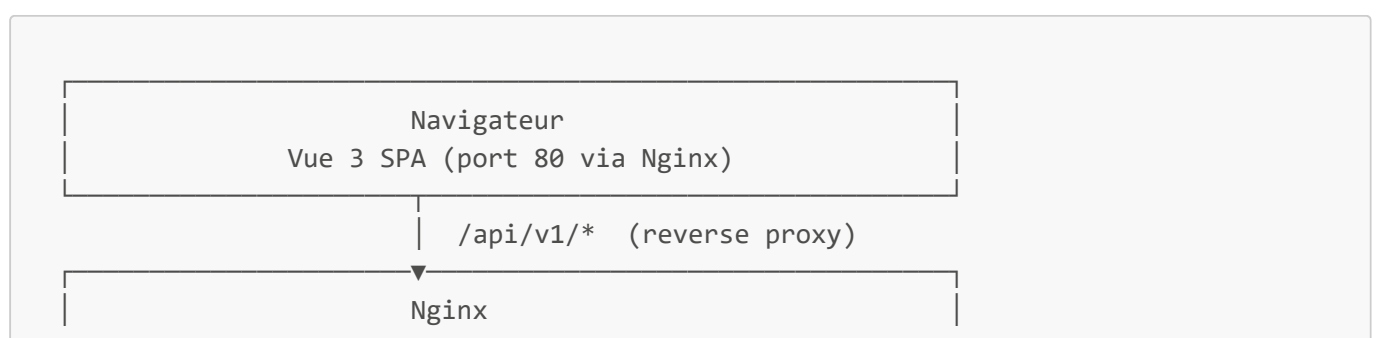
Frontend

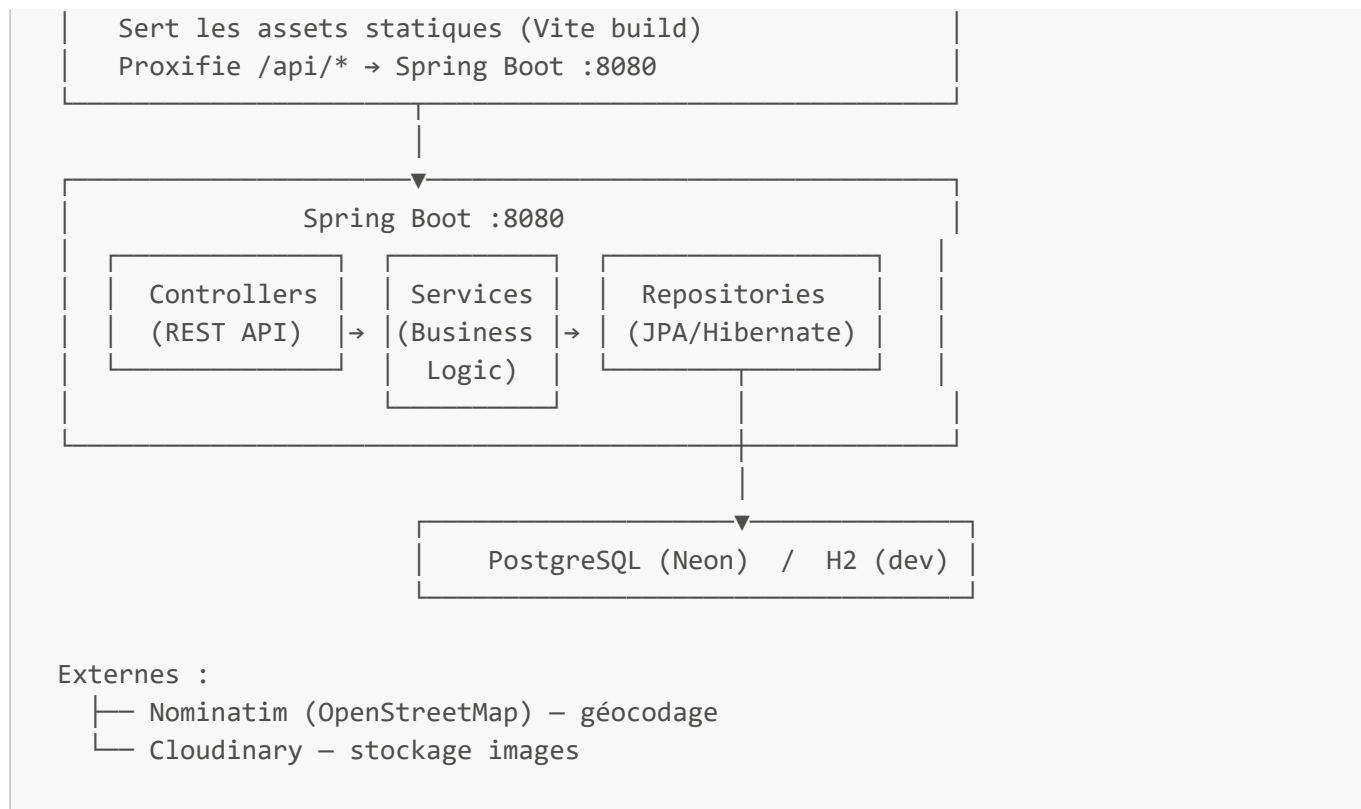
- **Framework** : Vue 3 (Composition API, `<script setup>`)
- **Build** : Vite 8
- **Router** : Vue Router 5
- **State** : Pinia 3
- **HTTP** : Axios 1.16
- **Cartes** : Leaflet 1.9 + `@vue-leaflet/vue-leaflet`
- **Icônes** : `lucide-vue-next`
- **Images** : Cloudinary (upload direct)
- **Linting** : ESLint + oxlint + Prettier

Infrastructure

- **Conteneurisation** : Docker + Docker Compose
- **Proxy** : Nginx (reverse proxy + SPA)
- **Orchestration** : Kubernetes (configs présentes)
- **Stockage images** : Cloudinary CDN
- **Base prod** : PostgreSQL Neon (serverless, SSL)

3. Architecture globale





4. Prérequis

- **Docker** ≥ 24 et **Docker Compose** ≥ 2.20
- **Java 21** (développement backend sans Docker)
- **Node.js 20** (développement frontend sans Docker)
- **Maven 3.9** (build backend sans Docker)
- Compte **Cloudinary** (upload d'images)
- Compte **Neon** (base PostgreSQL en production)

5. Démarrage rapide (développement)

Avec Docker (recommandé)

```
# Cloner le projet
git clone <url-du-repo>
cd immosn

# Copier et remplir le fichier d'environnement
cp .env.example .env.dev
# Remplir .env.dev (voir section Variables d'environnement)

# Démarrer en mode développement (H2 in-memory)
docker compose -f docker-compose.dev.yml --env-file .env.dev up --build

# Application disponible sur :
# Frontend : http://localhost
```

```
# Backend : http://localhost:8080
# H2 Console : http://localhost:8080/h2-console
```

Sans Docker — Backend seul

```
cd backend
cp src/main/resources/application.properties.example
src/main/resources/application.properties
# Remplir les variables d'environnement ou les mettre directement

mvn spring-boot:run
# API disponible sur http://localhost:8080/api/v1
```

Sans Docker — Frontend seul

```
cd frontend
cp .env.example .env.local
# VITE_API_URL=http://localhost:8080/api/v1

npm install
npm run dev
# Application disponible sur http://localhost:5173
```

6. Démarrage en production

```
# Remplir le fichier .env avec les valeurs de production
cp .env.example .env
# Éditer .env (voir section Variables d'environnement)

# Déployer avec le compose de production
docker compose --f docker-compose.local.yaml up -d --build

# Vérifier l'état des services
docker compose ps
docker compose logs -f backend
```

7. Variables d'environnement

Copier `.env.example` et remplir toutes les valeurs :

Base de données (PostgreSQL Neon)

Variable	Description	Exemple
DB_HOST	Hôte Neon (avec pooler)	ep-xxx.neon.tech
DB_NAME	Nom de la base	neondb
DB_USER	Utilisateur PostgreSQL	neondb_owner
DB_PASSWORD	Mot de passe PostgreSQL	npg_xxxx
DB_POOL_SIZE	Taille du pool Hikari	5
DDL_AUTO	Stratégie JPA DDL	update

Authentification JWT

Variable	Description	Valeur recommandée
JWT_SECRET	Clé secrète HMAC-SHA512 (min. 64 chars base64)	Générer : <code>openssl rand -base64 64</code>
JWT_EXPIRATION	Durée de vie du token (ms)	86400000 (24h)

Super Administrateur (bootstrap au démarrage)

Variable	Description
SUPER_ADMIN_EMAIL	Email du super admin créé automatiquement
SUPER_ADMIN_PASSWORD	Mot de passe (min. 8 chars, majuscule + chiffre + spécial)
SUPER_ADMIN_NOM	Nom complet
SUPER_ADMIN_TELEPHONE	Téléphone au format international

CORS et pagination

Variable	Défaut	Description
CORS_ORIGINS	http://localhost	Origines autorisées (séparées par virgule)
PAGE_DEFAULT_SIZE	10	Éléments par page (vue client)
PAGE_ADMIN_SIZE	20	Éléments par page (vue admin)
CONTRAT_EXPIRATION_CRON	0 5 0 * * *	Cron Spring d'expiration des contrats

Cloudinary (upload images)

Variable	Description
VITE_CLOUDINARY_CLOUD_NAME	Nom du cloud Cloudinary
VITE_CLOUDINARY_UPLOAD_PRESET	Preset d'upload non signé

Frontend

Variable	Description	Exemple
VITE_API_URL	URL de base de l'API	/api/v1 (prod) ou http://localhost:8080/api/v1 (dev)

8. Structure du projet

immosn/		
├─ backend/		# Spring Boot 4 / Java 21
│ └─ src/main/java/sn/immosn/backend/		
│ └─ annonce/		# Annonces immobilières
│ └─ data/entity/		# Annonce, TypeBienAnnonce, Commodity,
└─ AnnonceCommodity		
│ └─ data/repository/		# Repositories JPA
│ └─ service/		# AnnonceService, CommodityService,
└─ TypeBienService		
│ └─ auth/		# Authentification & utilisateurs
│ └─ data/entity/		# User, Role, RoleType, BlacklistedToken
│ └─ data/jwt/		# JwtTokenProvider,
└─ JwtAuthenticationFilter		
│ └─ data/repository/		
│ └─ service/		# AuthService, RoleDataInitializer
│ └─ lead/		# Gestion des prospects
│ └─ data/entity/		# Lead, StatutLead
│ └─ service/		
│ └─ visite/		# Demandes de visite
│ └─ data/entity/		# DemandeVisite, StatutDemandeVisite
│ └─ service/		
│ └─ discussion/		# Messagerie client-agent
│ └─ data/entity/		# Discussion, Message, SenderRole
│ └─ service/		
│ └─ contrat/		# Contrats de location/vente
│ └─ data/entity/		# Contrat, StatutContrat
│ └─ scheduler/		# ContratExpirationJob (cron quotidien)
│ └─ service/		
│ └─ favoris/		# Annonces favorites
│ └─ data/entity/		# AnnonceFavoris (clé composite)
│ └─ service/		
│ └─ signalement/		# Litiges sur contrats
│ └─ data/entity/		# Signalement, StatutSignalement
│ └─ service/		
│ └─ dashboard/		# Statistiques admin
│ └─ service/		
│ └─ location/		# Géolocalisation (Nominatim)
│ └─ GeoCodingService.java		
│ └─ LocationService.java		
│ └─ LocationController.java		
│ └─ GeocodeResponseDto.java		
│ └─ client/web/		# Couche présentation (Controllers + DTOs)
│ └─ annonce/		# AnnonceController + DTOs + Mapper

```

├── auth/                                # AuthController + DTOs + Mapper
├── lead/                                # LeadController + DTOs + Mapper
├── visite/                              # DemandeVisiteController + DTOs + Mapper
├── discussion/                          # DiscussionController + DTOs + Mapper
├── contrat/                             # ContratController + DTOs + Mapper
├── favoris/                             # FavorisController + DTOs
├── signalement/                         # SignalementController + DTOs + Mapper
├── dashboard/                           # DashboardController + DTO
├── config/
├──   SecurityConfig.java                # RBAC, CORS, JWT filter chain
├──   RestTemplateConfig.java
├── shared/
├──   exception/                         # GlobalExceptionHandler,
EntityNotFoundException
├──   response/                          # RestResponse<T>, PagedResponse<T>
├── Dockerfile                           # Multi-stage Maven build

├── frontend/                            # Vue 3 / Vite
├──   src/
├──   ├── main.js                        # Point d'entrée + Leaflet CSS
├──   ├── App.vue                        # Shell SPA (RouterView)
├──   ├── router/index.js                # Vue Router + guards de navigation
├──   ├── stores/
├──   ├──   authStore.js                 # Auth Pinia (token, user, rôle)
├──   ├──   favorisStore.js              # Favoris Pinia (Set<id>, toggle optimiste)
├──   ├──   toastStore.js                # Toasts Pinia (success/error/warning)
├──   ├── services/
├──   ├──   api.js                       # Axios instance + intercepteurs JWT
├──   ├──   authService.js
├──   ├──   annonceService.js
├──   ├──   leadService.js
├──   ├──   visiteService.js
├──   ├──   discussionService.js
├──   ├──   contratService.js
├──   ├──   favorisService.js
├──   ├──   signalementService.js
├──   ├──   dashboardService.js
├──   ├──   locationService.js
├──   ├──   cloundinaryService.js
├──   ├── components/
├──   ├──   AnnonceCard.vue              # Carte annonce (avec toggle favori)
├──   ├──   NavBar.vue
├──   ├──   LocationMap.vue              # Carte Leaflet (lecture + draggable)
├──   ├──   admin/                       # Composants admin réutilisables
├──   ├── layouts/
├──   ├──   ClientLayout.vue             # Layout public (navbar + footer)
├──   ├──   AdminLayout.vue             # Layout admin (sidebar)
├──   ├── views/
├──   ├──   auth/                        # Inscription, Connexion
├──   ├──   annonces/                   # Liste + Détail (public)
├──   ├──   profil/                     # Profil client
├──   ├──   discussions/                 # Messagerie client
├──   ├──   visites/                     # Visites client
├──   ├──   contrats/                   # Contrats client

```

```
| | | └─ signalements/      # Litiges client
| | |   └─ favoris/        # Favoris client
| | |     └─ admin/        # Toutes les vues admin
| | └─ Dockerfile
|
└─ nginx/
    │ └─ nginx.conf         # Reverse proxy (développement local)
    │ └─ nginx.vps.conf    # Reverse proxy production (SSL/TLS, rate
limiting, HSTS)
    │   └─ Dockerfile      # ARG NGINX_CONF – bascule auto vers
nginx.vps.conf en prod
    │
    └─ docker-compose.yml  # Production – VPS (Neon PostgreSQL,
NGINX_CONF=nginx.vps.conf)
    └─ docker-compose.dev.yml # Développement (H2 in-memory)
    └─ docker-compose.local.yaml # Développement Docker (PostgreSQL locale)
    └─ .env.example          # Template variables d'environnement
    └─ README.md
```

9. Modèles de données

Diagramme des entités principales



Entités détaillées

User

id	Long	PK
nomComplet	String	NOT NULL
email	String	UNIQUE, NOT NULL
telephone	String	NOT NULL
adresse	String	nullable
photo	String	nullable (URL)
password	String	BCrypt
creationDate	LocalDateTime	@PrePersist
isArchived	boolean	default false
roles	Set<Role>	ManyToMany (user_roles)

Annonce

id	Long	PK
libelle	String	NOT NULL
description	TEXT	nullable
nbrPieces	Integer	NOT NULL
surface	Double	NOT NULL
prix	BigDecimal	NOT NULL (precision 10, scale 2)
adresse	String	nullable
region	String	nullable (toujours "Dakar")
departement	String	NOT NULL
quartier	String	NOT NULL
latitude	Double	nullable (géocodé par Nominatim)
longitude	Double	nullable (géocodé par Nominatim)
typeBien	→TypeBien	NOT NULL
annonceCommodites	→[]	cascade
images	List<String>	table annonce_images (URLs Cloudinary)
isArchived	boolean	default false
createdAt	LocalDateTime	@PrePersist
updatedAt	LocalDateTime	@PreUpdate

Lead

id	Long	PK
client	→User	NOT NULL
annonce	→Annonce	NOT NULL
visite	→DemandeVisite	nullable
statut	StatutLead	EN_COURS CONVERTI ABANDONNE
noteAdmin	TEXT	nullable
createdAt / updatedAt		

DemandeVisite

id	Long	PK
client	→User	NOT NULL
annonce	→Annonce	NOT NULL
dateVisite	LocalDateTime	NOT NULL
statut	StatutDemandeVisite	EN_ATTENTE ACCEPTEE REFUSEE ANNULEE TERMINEE
commentaire	TEXT	nullable
isArchived	boolean	default false
createdAt / updatedAt		

Discussion + Message

Discussion:

```
id, client →User, annonce →Annonce
UNIQUE (client_id, annonce_id)
messages →[]Message
createdAt
```

Message:

```
id, contenu TEXT, senderRole (CLIENT|ADMIN)
isRead boolean, discussion →Discussion, createdAt
```

Contrat

id	Long	PK
client	→User	NOT NULL
annonce	→Annonce	NOT NULL
lead	→Lead	nullable
dateDebut	LocalDate	NOT NULL
dateFin	LocalDate	nullable
montant	BigDecimal	NOT NULL (precision 12, scale 2)
statut	StatutContrat	EN_ATTENTE ACTIF EXPIRE RESILIE EN_ATTENTE_RESILIATION PROLONGATION_EN_ATTENTE
documentUrl	String	nullable
notes	TEXT	nullable
createdAt / updatedAt		

Signalement

```
id, contrat →Contrat, client →User
contenu TEXT, statut (OUVERT|EN_COURS|RESOLU|FERME)
isRead boolean, reponseAdmin TEXT, createdAt
```

10. API REST — Endpoints

Toutes les routes sont préfixées par `/api/v1`.

La réponse standard est :

```
{
  "success": true,
  "message": "...",
  "data": { ... },
  "timestamp": "2025-01-01T12:00:00"
}
```

La réponse paginée est :

```
{
  "data": [...],
  "pagination": {
    "page": 0, "size": 10,
    "totalElements": 100, "totalPages": 10,
    "first": true, "last": false
  }
}
```

Auth — `/api/v1/auth`

Méthode	Route	Accès	Description
POST	<code>/register</code>	Public	Créer un compte CLIENT
POST	<code>/login</code>	Public	Connexion — retourne JWT
POST	<code>/logout</code>	Authentifié	Invalider le token (blacklist)
GET	<code>/profile</code>	Authentifié	Profil de l'utilisateur courant
PUT	<code>/profile</code>	Authentifié	Modifier son profil
POST	<code>/admin</code>	SUPER_ADMIN	Créer un compte ADMIN
GET	<code>/admins</code>	SUPER_ADMIN	Lister les admins (paginé)
PATCH	<code>/admins/{id}/archive</code>	SUPER_ADMIN	Archiver un admin
PATCH	<code>/admins/{id}/restore</code>	SUPER_ADMIN	Restaurer un admin
PATCH	<code>/admins/{id}/revoke</code>	SUPER_ADMIN	Révoquer le rôle ADMIN

Corps POST `/register` :

```
{
  "nomComplet": "Moussa Diallo",
  "email": "moussa@example.com",
  "telephone": "+221701234567",
  "motDePasse": "MonMotDePasse123!"
}
```

Réponse POST /login :

```
{
  "data": {
    "accessToken": "eyJhbGciOi...",
    "tokenType": "Bearer",
    "roles": ["CLIENT"]
  }
}
```

Annonces — /api/v1/annonces

Méthode	Route	Accès	Description
GET	/	Public	Lister les annonces actives (paginé)
GET	/ {id}	Public	Détail d'une annonce
POST	/search	Public	Recherche avancée (filtres combinables)
GET	/admin	ADMIN	Toutes les annonces (y compris archivées)
POST	/	ADMIN	Créer une annonce
PUT	/ {id}	ADMIN	Modifier une annonce
DELETE	/ {id}	ADMIN	Archiver (soft delete)
PATCH	/ {id}/restore	ADMIN	Restaurer une annonce archivée

Paramètres GET / (paginé) :

```
?page=0&size=10&sort=createdAt&direction=DESC
```

Corps POST /search :

```
{
  "libelle": "Villa",
  "typeBienId": 1,
  "prixMin": 50000000,
}
```

```
"prixMax": 2000000000,  
"departement": "Dakar",  
"quartier": "Almadies",  
"nbrPiecesMin": 3,  
"page": 0,  
"size": 9,  
"sortBy": "prix",  
"sortDir": "ASC"  
}
```

Corps POST / (création) :

```
{  
  "libelle": "Villa moderne à Almadies",  
  "description": "...",  
  "nbrPieces": 5,  
  "surface": 250.0,  
  "prix": 1500000000,  
  "adresse": "12 Rue des Ambassadeurs",  
  "departement": "Dakar",  
  "quartier": "Almadies",  
  "typeBienId": 1,  
  "commoditeIds": [1, 3, 5],  
  "images": ["https://res.cloudinary.com/..."]  
}
```

Référentiels

Types de bien — /api/v1/types-bien

Méthode	Route	Accès	Description
GET	/	Public	Tous les types
GET	/paged	Public	Types paginés
POST	/	ADMIN	Créer un type
PUT	/ {id}	ADMIN	Modifier
DELETE	/ {id}	ADMIN	Supprimer

Commodités — /api/v1/commodites

Méthode	Route	Accès	Description
GET	/	Public	Toutes les commodités
GET	/paged	Public	Commodités paginées

Méthode	Route	Accès	Description
POST	/	ADMIN	Créer
PUT	/ {id}	ADMIN	Modifier
DELETE	/ {id}	ADMIN	Supprimer

Leads — /api/v1/leads (ADMIN uniquement)

Méthode	Route	Description
POST	/	Créer un lead
GET	/	Lister (filtrable par ?statut=EN_COURS)
GET	/ {id}	Détail
PUT	/ {id}/status	Modifier le statut (+ note admin optionnelle)
PUT	/ {id}/note	Mettre à jour la note admin uniquement
GET	/ {id}/historique	Historique des transitions (Audit Trail)

Corps POST / :

```
{
  "clientId": 42,
  "annonceId": 7,
  "visiteId": 3,
  "noteAdmin": "Client très motivé"
}
```

Corps PUT / {id}/status :

```
{
  "statut": "CONVERTI",
  "noteAdmin": "Contrat signé"
}
```

Visites — /api/v1/visites

Méthode	Route	Accès	Description
POST	/	CLIENT	Créer une demande de visite
GET	/client	CLIENT	Mes visites (filtrable par statut)
GET	/admin	ADMIN	Toutes les visites

Méthode	Route	Accès	Description
GET	<code>/{{id}}</code>	Authentifié	Détail d'une visite (client : les siennes / admin : toutes)
PUT	<code>/{{id}}/status</code>	Authentifié	Modifier le statut (ADMIN : accepter/refuser — CLIENT : annuler)
PUT	<code>/{{id}}/date</code>	ADMIN	Replanifier la date
PUT	<code>/{{id}}/modifier</code>	CLIENT	Modifier sa demande (date/commentaire, si EN_ATTENTE)
PUT	<code>/{{id}}/cloture</code>	ADMIN	Clôturer une visite ACCEPTEE (SANS_SUITE ou AVEC_CONTRAT)
DELETE	<code>/{{id}}</code>	CLIENT	Annuler sa demande
GET	<code>/{{id}}/historique</code>	ADMIN	Historique des transitions (Audit Trail)

Corps POST / :

```
{
  "annonceId": 7,
  "dateVisite": "2025-03-15T10:00:00",
  "commentaire": "Je souhaite visiter samedi matin"
}
```

Discussions & Messages — `/api/v1/discussions`

Méthode	Route	Accès	Description
POST	<code>/</code>	CLIENT	Créer ou récupérer une discussion (idempotent)
GET	<code>/client</code>	CLIENT	Mes discussions (triées par dernière activité)
GET	<code>/admin</code>	ADMIN	Toutes les discussions
GET	<code>/{{id}}/messages</code>	Authentifié	Messages (marque comme lus)
POST	<code>/{{id}}/messages</code>	Authentifié	Envoyer un message

Corps POST / :

```
{
  "annonceId": 7,
  "premierMessage": "Bonjour, je suis intéressé par ce bien."
}
```

Contrats — `/api/v1/contrats`

Méthode	Route	Accès	Description
POST	/	ADMIN	Créer un contrat
GET	/client	CLIENT	Mes contrats (filtrable par statut)
GET	/admin	ADMIN	Tous les contrats
GET	/{\$id}	Authentifié	Détail (client : les siens / admin : tous)
PUT	/{\$id}	ADMIN	Modifier (statut, dates, montant, document)
PUT	/{\$id}/resiliation	CLIENT	Soumettre une demande de résiliation
PUT	/{\$id}/resiliation/accepter	ADMIN	Accepter la demande → statut RESILIE
PUT	/{\$id}/resiliation/refuser	ADMIN	Refuser la demande → statut ACTIF
PUT	/{\$id}/prolongation	CLIENT	Soumettre une demande de prolongation
PUT	/{\$id}/prolongation/accepter	ADMIN	Accepter la prolongation → nouvelle dateFin
PUT	/{\$id}/prolongation/refuser	ADMIN	Refuser la prolongation → statut ACTIF
GET	/{\$id}/historique	ADMIN	Historique des transitions (Audit Trail)

Corps POST / (ADMIN) :

```
{
  "clientId": 42,
  "annonceId": 7,
  "leadId": 3,
  "dateDebut": "2025-02-01",
  "dateFin": "2026-01-31",
  "montant": 250000,
  "documentUrl": "https://...",
  "notes": "Contrat de location annuel"
}
```

Favoris — /api/v1/favoris (CLIENT uniquement)

Méthode	Route	Description
POST	/{\$annonceId}/toggle	Ajouter/retirer des favoris
GET	/	Mes favoris (paginé, 12/page)
GET	/ids	Tous les IDs favoris (sans limite)
GET	/{\$annonceId}/check	Vérifier si une annonce est en favori

Signalements — </api/v1/signalements>

Méthode	Route	Accès	Description
POST	/	CLIENT	Signaler un problème sur un contrat
GET	/client	CLIENT	Mes signalements
GET	/admin	ADMIN	Tous les signalements
PUT	/{id}/status	ADMIN	Modifier statut + réponse
PUT	/{id}/read	ADMIN	Marquer comme lu

Dashboard — </api/v1/admin/dashboard> (ADMIN)

Méthode	Route	Description
GET	/stats	Toutes les statistiques en un appel

Réponse [/stats](/api/v1/admin/dashboard/stats) :

```
{
  "data": {
    "totalAnnonces": 45,
    "annoncesActives": 38,
    "totalClients": 120,
    "totalAdmins": 3,
    "totalContrats": 31,
    "contratsActifs": 18,
    "totalVisites": 89,
    "visitesEnAttente": 12,
    "visitesAujourd'hui": 2,
    "totalSignalements": 5,
    "signalementsOuverts": 2,
    "totalLeads": 67,
    "leadsEnCours": 23,
    "leadsConvertis": 31,
    "leadsAbandonnes": 13,
    "tauxConversionLeads": 70.45,
    "totalDiscussions": 93,
    "activitesRecentes": [
      {
        "type": "VISITE",
        "titre": "Visite : Villa F5 - Almadies",
        "description": "Aminata Diallo",
        "statut": "EN_ATTENTE",
        "createdAt": "2024-01-15T11:30:00"
      }
    ]
  }
}
```

Audit Trail — Historique métier (ADMIN)

Chaque module métier expose un endpoint de traçabilité enregistrant toutes les transitions de statut et actions effectuées.

Méthode	Route	Description
GET	/contrats/{id}/historique	Historique des transitions d'un contrat
GET	/leads/{id}/historique	Historique des transitions d'un lead
GET	/visites/{id}/historique	Historique des transitions et reprogrammations d'une visite

Réponse paginée (même structure pour les trois endpoints) :

```
{
  "content": [
    {
      "id": 1,
```

```
        "ancienStatut": "EN_ATTENTE",
        "nouveauStatut": "ACTIF",
        "auteurId": 5,
        "auteurEmail": "admin@immosn.sn",
        "action": "ACTIVATION",
        "commentaire": null,
        "createdAt": "2024-02-01T10:00:00"
    }
  ],
  "page": 0, "size": 20, "totalElements": 3, "totalPages": 1, "last": true
}
```

VisiteHistoryDto inclut en plus ancienneDateVisite et nouvelleDateVisite pour tracer les reprogrammations.

Géolocalisation — /api/v1/locations (Public / ADMIN)

Méthode	Route	Accès	Description
GET	/departements	Public	Liste des départements disponibles
GET	/quartiers?departement={nom}	Public	Quartiers d'un département
GET	/geocode? departement=&quartier=&adresse=	Public	Coordonnées GPS via Nominatim (prévisualisation carte)

Départements disponibles (région Dakar uniquement) :

- Dakar, Pikine, Guédiawaye, Rufisque, Keur Massar

11. Sécurité et authentification

Flux d'authentification JWT

1. POST /auth/login { email, motDePasse }

↳ Spring Security valide les credentials (BCrypt)

↳ JwtTokenProvider génère un token HMAC-SHA512 (24h)

↳ Retourne { accessToken, tokenType: "Bearer" }
2. Requêtes authentifiées :

↳ Header : Authorization: Bearer <token>

↳ JwtAuthenticationFilter extrait et valide le token

↳ Vérifie que le token n'est pas blacklisté

↳ Injecte l'utilisateur dans le SecurityContext
3. POST /auth/logout

↳ Ajoute le token à la table BlacklistedToken

↳ Token invalide pour toutes les requêtes suivantes

RBAC — Matrice des permissions

Ressource	Public	CLIENT	ADMIN	SUPER_ADMIN
Annonces (lecture)	✓	✓	✓	✓
Annonces (écriture)	✗	✗	✓	✓
Leads	✗	✗	✓	✓
Visites	✗	✓ (les siennes)	✓ (toutes)	✓
Discussions	✗	✓ (les siennes)	✓ (toutes)	✓
Contrats	✗	✓ (les siens)	✓ (tous)	✓
Favoris	✗	✓	✗	✓
Signalements	✗	✓ (les siens)	✓ (tous)	✓
Dashboard	✗	✗	✓	✓
Gestion admins	✗	✗	✗	✓
/locations/geocode	✓	✓	✓	✓

Génération de la clé JWT

```
openssl rand -base64 64
# Coller la valeur dans JWT_SECRET du fichier .env
```

12. Flux métier

Cycle de vie d'un prospect (Lead)

```
CLIENT visite une annonce
├── Envoie un message (Discussion)
│   └── ADMIN répond via messagerie
└── Demande une visite (DemandeVisite)
    ├── statut: EN_ATTENTE
    │   ├── ADMIN accepte → ACCEPTEE
    │   ├── Visite effectuée → TERMINEE
    │   └── ADMIN crée un Lead (client + annonce + visite)
    │       ├── statut: EN_COURS
    │       │   ├── ADMIN crée un Contrat → Lead: CONVERTI
    │       │   └── ADMIN abandonne → Lead: ABANDONNE
```

Cycle de vie d'un contrat

```

ADMIN crée Contrat → statut: EN_ATTENTE
├── ADMIN active → statut: ACTIF
│   ├── [Cron quotidien 00h05] dateFin dépassée → EXPIRE
│   ├── CLIENT demande résiliation
│   │   └── EN_ATTENTE_RESILIATION → [ADMIN] RESILIE
│   └── CLIENT demande prolongation
│       └── PROLONGATION_EN_ATTENTE → [ADMIN] ACTIF (nouvelle
dateFin)

```

Flux de messagerie

```

CLIENT ouvre une annonce → clique "Contacter"
├── POST /discussions { annonceId, premierMessage }
│   ├── Idempotent: même (client, annonce) → même discussion
│   └── ADMIN voit la discussion dans le panel admin
│       ├── Répond → Message(senderRole: ADMIN)
│       │   └── CLIENT reçoit la réponse (isRead: false)
│       │       └── Marque comme lu à l'ouverture

```

Géocodage des annonces

```

ADMIN remplit formulaire → sélectionne département + quartier
├── Frontend appelle GET /locations/geocode (prévisualisation carte)
│   ├── Backend appelle Nominatim (OpenStreetMap)
│   └── Retourne latitude/longitude pour aperçu
├── ADMIN soumet l'annonce (POST /annonces)
│   ├── Backend régéocode automatiquement (source de vérité)
│   │   └── Coordonnées sauvegardées en base
│   └── Carte affichée sur la page de détail

```

13. Frontend — Vue.js

Stores Pinia

authStore

```
// État
token          // JWT depuis localStorage
user           // { id, nomComplet, email, roles, ... }
role           // 'CLIENT' | 'ADMIN' | 'SUPER_ADMIN'

// Getters
isAuthenticated
isAdmin        // role === ADMIN || SUPER_ADMIN

// Actions
login(email, motDePasse) // POST /auth/login → set token + redirect
logout()              // POST /auth/logout → clear + redirect /connexion
fetchProfile()        // GET /auth/profile → refresh user data
init()                // Appelé au montage App.vue pour restaurer la session
```

favorisStore

```
// État
favorisIds      // Set<Long> – chargé une fois par session

// Actions
loadFavoris()          // GET /favoris/ids → remplit le Set
toggle(annonceId)      // Optimiste : toggle immédiat + rollback si erreur
isFavori(annonceId)    // Boolean – accès synchrone
reset()                // Vidé à la déconnexion
```

Intercepteurs Axios (`services/api.js`)

Requête — Ajoute automatiquement le token JWT :

```
headers.Authorization = `Bearer ${token}`
```

Réponse — Gestion centralisée des erreurs :

- 401 → clear localStorage + redirect `/connexion`
- Enrichit les erreurs avec `userMessage` pour l'affichage UI

Guards de navigation (Vue Router)

```
// Pour chaque route avec meta.requiresAuth = true
if (!isAuthenticated) → redirect /connexion

// Pour les routes admin (meta.role = 'ADMIN')
if (!isAdmin) → redirect /annonces
```

Pattern de service (exemple)

```
// Appel
const res = await annonceService.getAll(0, 10)
// res.data.data      → tableau d'annonces
// res.data.pagination → { page, size, totalElements, totalPages }

// Toutes les réponses suivent RestResponse<T> ou PagedResponse<T>
```

14. Géolocalisation

Architecture

```
Vue (formulaire) → GET /api/v1/locations/geocode (ADMIN auth)
                  ↳ GeoCodingService → Nominatim OpenStreetMap
                  ↳ Retourne { latitude, longitude } (prévisualisation)

Soumission formulaire → Spring Boot régéocode automatiquement
                      ↳ Sauvegarde les coordonnées définitives en base
```

Données disponibles (région Dakar)

Département	Exemples de quartiers
Dakar	Almadies, Plateau, Médina, Yoff, Ngor, Ouakam, Mermoz, Point E...
Pikine	Pikine Est, Pikine Ouest, Thiaroye, Yeumbeul, Diamaguène...
Guédiawaye	Golf Sud, Hamo, Sam Notaire, Wakhinane Nimzatt...
Rufisque	Rufisque Est/Nord/Ouest, Bargny, Diamniadio, Sébikotane
Keur Massar	Keur Massar Nord/Sud, Malika, Jaxaay-Parcelles...

Composant `LocationMap.vue`

```
<!-- Lecture seule -->
<LocationMap :latitude="annonce.latitude" :longitude="annonce.longitude"
height="260px" />

<!-- Draggable (formulaire admin) -->
<LocationMap
  :latitude="form.latitude"
  :longitude="form.longitude"
  :draggable="true"
  height="280px"
  @update:latitude="v => form.latitude = v"
```

```
@update:longitude="v => form.longitude = v"
/>
```

15. Déploiement Docker

Fichiers disponibles

Fichier	Usage	Base de données
<code>docker-compose.yml</code>	Production — VPS (nginx.vps.conf)	PostgreSQL Neon
<code>docker-compose.dev.yml</code>	Développement local	H2 in-memory
<code>docker-compose.local.yml</code>	Développement Docker	PostgreSQL locale

Commandes utiles

```
# Développement
docker compose -f docker-compose.dev.yml up --build

# Production local
docker compose -f docker-compose.local.yml up -d --build

# Production
docker compose up -d --build

# Logs backend
docker compose logs -f backend

# Logs nginx
docker compose logs -f nginx

# Accéder au shell du backend
docker compose exec backend sh

# Rebuild un seul service
docker compose up --build backend -d
```

Health check

Le backend expose `GET /actuator/health` (sans requête DB) — réponse en < 1 s même après cold start Neon.

Docker Compose attend que le backend soit `healthy` avant de démarrer Nginx.

```
healthcheck:
  test: ["CMD-SHELL", "wget -q -O /dev/null http://127.0.0.1:8080/actuator/health || exit 1"]
  interval: 30s
```



```
timeout: 10s
retries: 5
start_period: 90s
```

16. Configuration Nginx

Rôles de Nginx

1. **Serveur de fichiers statiques** — Sert le build Vite (/app/dist)
2. **SPA fallback** — Toutes les routes inconnues → `index.html` (Vue Router)
3. **Reverse proxy** — `/api/*` → `http://backend:8080`
4. **Cache assets** — 1 an pour les fichiers hachés Vite (`.js`, `.css`, `.woff2`)
5. **Headers de sécurité** — `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection`
6. **Compression** — gzip activé (min. 1 Ko, niveau 5)
7. **H2 Console bloquée** — `/h2-console` renvoie 403 en production

Configuration type

```
location /api/ {
    proxy_pass http://backend:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

location / {
    root /usr/share/nginx/html;
    try_files $uri /index.html; # SPA fallback
}

location ~* \.(js|css|woff2|png|jpg|svg)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

Codes d'erreur HTTP

Code	Cas
200	Succès
201	Ressource créée
204	Succès sans contenu (archive, restore)
400	Paramètre invalide / validation échouée
401	Token absent ou invalide

Code	Cas
403	Rôle insuffisant
404	Ressource non trouvée
409	Conflit (email déjà utilisé)
422	Règle métier violée (ex : archiver un contrat actif)
500	Erreur interne non gérée

Conventions de code

Backend

- **Packages** : `sn.immosn.backend.<domaine>.<couche>`
- **DTOs** : `{Entité}CreateRequestDto`, `{Entité}UpdateRequestDto`, `{Entité}ResponseDto`
- **Services** : interface + `Impl` séparés
- **Réponses** : toujours wrappées dans `RestResponse<T>` ou `PagedResponse<T>`
- **Exceptions** : `EntityNotFoundException` (404), `EntityExistException` (409), `IllegalStateException` (422)
- **Soft delete** : champ `isArchived` sur toutes les entités supprimables
- **Transactions** : `@Transactional` sur les méthodes de service mutantes

Frontend

- **Composants** : `<script setup>` (Composition API)
- **Services** : une fonction par endpoint, retournent la Promise Axios
- **Stores** : Pinia avec `defineStore`, nommés `use{Nom}Store`
- **Vues admin** : suffixe `AdminView` ou préfixe `Admin`
- **Gestion d'erreurs** : `err.userMessage || err.response?.data?.message` pour l'affichage toast